Cloud Application and Development

Cloud Backup Verification and Recovery System

Final Project Report · Project ID P26 · Category: Fault Tolerance & Resilience

Student name	Aryan Raj
SAP / Roll number	500126035
Section	B4
Email	aryanxraj21@gmail.com
Project type	Individual project
Submitted to	Faculty, Cloud Application and Development
Date	12 October 2026
Source code	Git repository: cloud-backup-recovery (14 commits)

1. Abstract

This project builds a cloud service that protects application data with **verifiable backups** and **measured recovery**. Every record is stored with a SHA-256 checksum; backups are written to object storage with a whole-file checksum. A fault-injection endpoint deliberately deletes and silently corrupts records, breaks the storage dependency (full outage or 50% transient errors) and corrupts a backup file. The system detects every injected fault, restores data through a queued, atomic, self-verifying recovery job, and uses retry with exponential backoff, a circuit breaker, idempotency keys, job re-queueing and backup failover to survive dependency failures.

Key results: all 200 damaged records in the largest test were detected and repaired in **29 ms**; a 6-second storage outage was survived with the restore job automatically resuming (RTO 7.6 s) while **52/52** read requests kept succeeding; a corrupted backup was rejected and the restore failed over to the previous intact backup. After every scenario a SHA-256 fingerprint of the whole dataset matched the pre-failure baseline, proving no data was corrupted. Load testing at 5, 25 and 100 concurrent users processed 7,500 requests with **0% errors**.

2. Problem Statement and Objectives

Backups are only useful if they can be trusted and restored quickly. Many teams discover a backup is incomplete or corrupted only during an incident. The assigned problem is to design, develop, deploy and evaluate a cloud application that creates backups, deliberately deletes or corrupts records, and demonstrates restoration while measuring recovery behaviour, showing cloud-specific behaviour rather than plain CRUD.

- Create consistent, checksummed backups in object storage, safely retryable (idempotent).
- Detect deleted records, silently corrupted records and corrupted backup files.
- Restore atomically (no half-restored state) and verify the result automatically.
- Keep working when the storage dependency fails: retry, circuit breaker, queue, graceful degradation, failover.
- Measure detection time, recovery time (RTO), retries and success/failure counts; load test three levels.
- Secure everything with JWT + role-based access and keep structured logs of all events.

3. Requirement Compliance Matrix

#	Requirement (from allocation)	How it is met	Section
1	Backups, delete/corrupt, restore, measure	POST /backups, POST /chaos, POST /restores; recovery_time_ms	6, 8
2	≥ 5 REST endpoints	15 endpoints (auth, records, backups, verify, restores, chaos, metrics, logs, health)	7
3	Proper data model	5 tables: users, records, backups, restore_jobs, events	5
4	Authentication / authorization	JWT (HS256, expiry) + RBAC viewer < operator < admin	7.2
5	Deployed on cloud/deployable env.	Docker image, Render blueprint, docker-compose with S3-compatible MinIO	12
6	Logs of events/failures/actions	JSON log file + events table, GET /logs	10
7	≥ 2 metrics in report	Recovery time, detection time, retries, attempts, latency, throughput, error rate	8, 9

#	Requirement (from allocation)	How it is met	Section
8	Load test low/medium/high	5 / 25 / 100 concurrent users, 7,500 requests	9
9	Architecture diagram	API, services, DB, queue, object storage, users	4
10	API docs + report + screenshots	docs/API.md, Swagger /docs, this report, Section 11	7, 11
11	Deliberate failure	Deletion, silent corruption, storage outage, 50% flaky storage, corrupted backup	8
12	Recovery mechanism	Retry+backoff, circuit breaker, queue, idempotency, backup, failover, graceful degradation	6.3
13	Data not corrupted after failure	Dataset SHA-256 fingerprint identical to baseline after every scenario	8.5
14	Failure + recovery in logs	failure_injected, damage_detected, retry, circuit_state_change, restore_deferred, backup_failover, restore_completed	10
15	Measure recovery time / retries / counts	recovery_time_ms, storage_retries, attempts, http_success/failed_total	8
C9	Git repo ≥ 8 commits	14 meaningful commits	13

4. Cloud Architecture

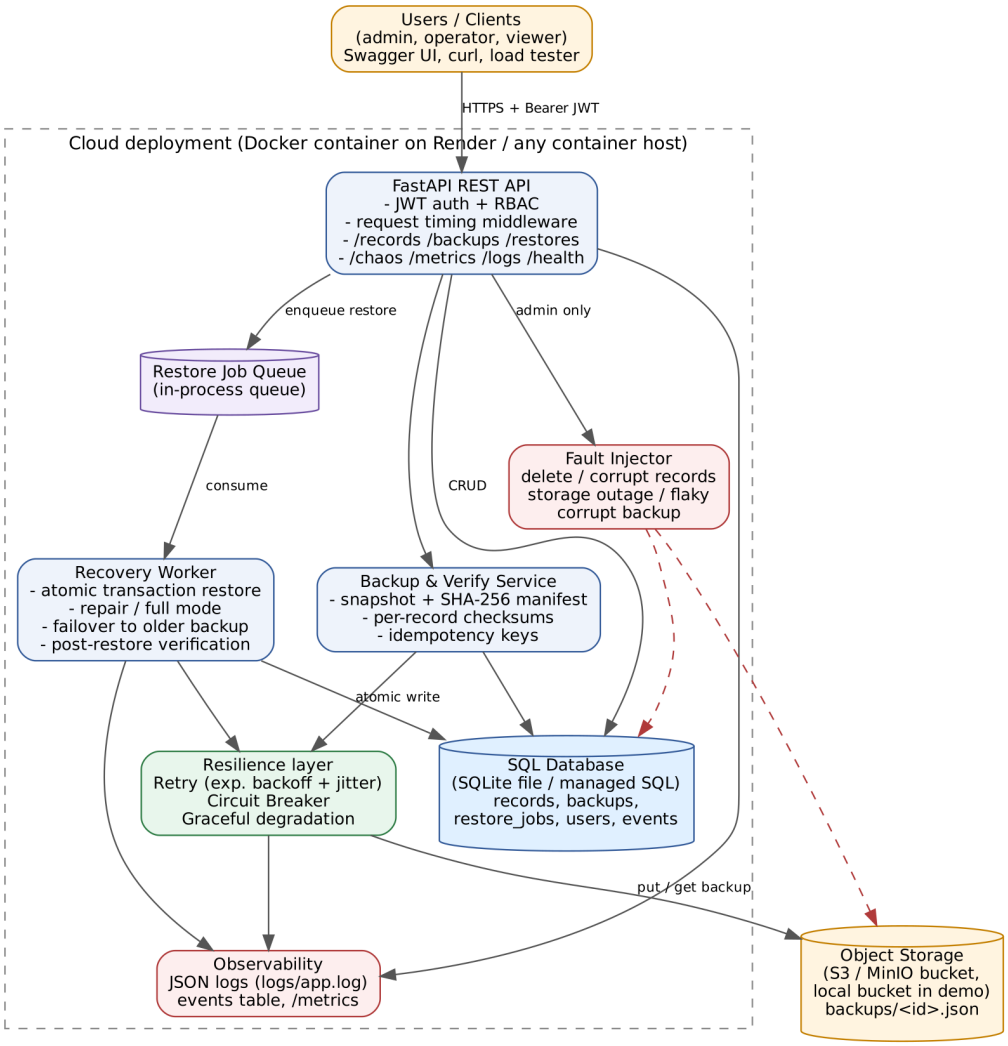


Figure 1 – Cloud architecture: users, API, services, queue, database, object storage, observability

Cloud / architectural components used (requirement: at least 3):

Component	Role in the system	Cloud equivalent
Containerised REST API	Stateless FastAPI service, JWT auth, request metrics	Render web service / AWS ECS / Cloud Run
Object storage	Durable backup files backups/<id>.json, written atomically	AWS S3 / MinIO (S3 API); local bucket in demo
SQL database	Records, backup catalogue, jobs, users, events	SQLite file on volume; Amazon RDS / Cloud SQL in production
Job queue + worker	Asynchronous restores, re-queue with backoff	In-process queue; SQS / Cloud Tasks in production
Circuit breaker + retry	Protects against storage failures, fails fast when open	Resilience pattern (like AWS SDK retries / Envoy)
Observability	JSON logs, events table, /metrics, /health	CloudWatch / Cloud Logging, health checks

5. Technology Choices and Data Model

Stack: Python 3.12, FastAPI (auto OpenAPI docs), SQLite (WAL mode), PyJWT, boto3 for S3, httpx for load testing, Docker. **Why SQL:** the entities are relational (a restore job references a backup; events reference jobs), and a restore must be **atomic**: either every repaired row is written or none. A transactional SQL database gives BEGIN/COMMIT/ROLLBACK for free, which NoSQL stores without multi-document transactions do not. Backup files themselves are large, immutable blobs, so they go to **object storage**, which is cheaper, more durable and decoupled from the database it protects (a backup on the same disk as the data is not a backup).

Entity	Key fields	Purpose
users	id, username, password_hash (PBKDF2), role	Login and RBAC
records	id, name, payload (JSON), checksum (SHA-256), updated_at	The protected business data
backups	id, created_at, created_by, status, record_count, size_bytes, object_key, manifest_checksum, idempotency_key, duration_ms, last_verify_status	Backup catalogue; object_key points to storage
restore_jobs	id, backup_id (FK), used_backup_id, mode, fallback, status, attempts, storage_retries, records_restored, recovery_time_ms, verified, error	Recovery jobs and their measurements
events	id, ts, level, actor, event, details (JSON)	Audit / failure / recovery log

6. How the System Works

6.1 Backup

A backup reads all records, writes a JSON manifest (records + their checksums) to object storage via an atomic write (temp file then rename) and stores the SHA-256 of the whole file in the catalogue. An *Idempotency-Key* header guarantees that a client retrying the same request gets the same backup instead of a duplicate (verified in the experiment: two calls returned **bkp_20261012092836_d9b89b** both times).

6.2 Verification (detection)

Verify first re-downloads the backup and checks its SHA-256 (detects a corrupted backup). Then for every record in the backup it checks the live DB: **missing** (deleted), **corrupted** (data no longer matches its own checksum = silent corruption / bit rot), or legitimately **changed** since the backup.

6.3 Recovery and failure handling

Mechanism	Where	What it does
Queue	POST /restores → worker	Restore runs asynchronously; API returns 202 immediately and stays responsive
Retry + exponential backoff + jitter	storage get/put	Up to 4 tries, 0.1→0.2→0.4 s (+jitter) for transient 503s
Circuit breaker	storage client	Opens after 3 consecutive failures, fails fast for 3 s, then half-open probe
Job re-queue	worker	If storage is down, job → RETRY_SCHEDULED and re-runs after 1, 2, 4, 8 s (max 6 attempts)
Atomic transaction	worker	All repairs in one DB transaction; error = rollback, never half-restored
Repair mode (idempotent)	worker	Only rewrites missing/corrupted rows; running it twice is harmless
Backup failover	worker	If the chosen backup fails its checksum, use the newest older intact backup
Graceful degradation	API	Reads/writes keep working while storage is down; /health reports 'degraded'

Mechanism	Where	What it does
Post-restore verification	worker	Full integrity scan; job is COMPLETED only if verified = true

7. REST API and Security

7.1 Endpoints

Method & path	Role	Description
POST /auth/login	public	Returns JWT
POST /records	operator	Create record (checksum stored)
GET /records, GET /records/{id}	viewer	List / get with integrity flag
POST /backups	operator	Create backup (Idempotency-Key)
GET /backups, GET /backups/{id}	viewer	Backup catalogue
POST /backups/{id}/verify	operator	Detect missing / corrupted records and backup corruption
POST /restores, GET /restores/{id}, GET /restores	operator / viewer	Queue restore, poll status
POST /chaos	admin	Inject failures
GET /metrics	viewer	Metrics
GET /logs	admin	Event logs
GET /health	public	Health and breaker state

Full request/response documentation is in *docs/API.md* and interactively at */docs* (Swagger UI).

7.2 Authentication and authorization

Users log in with a password (stored as salted PBKDF2-SHA256, 100k iterations) and receive a JWT signed with HS256 that expires after 1 hour. Every protected endpoint checks the token and the role rank viewer < operator < admin. Destructive fault injection and log access are admin-only. Tested: a request without a token returns 401 and a viewer calling /chaos returns 403 (Section 11).

8. Failure Experiments and Results

All experiments are automated in *scripts/failure_experiment.py* on a dataset of 1,000 records (results/failure_experiment.json). Before the faults, a SHA-256 fingerprint of every (id, checksum, payload) was taken as the baseline.

8.1 Scenario A – deleted and silently corrupted records

Damaged	Missing found	Corrupted found	Detect (ms)	Recovery (ms)	Restored	Verified	Identical to baseline
10	5	5	15.8	25.7	10	True	True
50	25	25	14.8	28.3	50	True	True
100	50	50	23.1	31.2	100	True	True
200	100	100	14.8	28.8	200	True	True

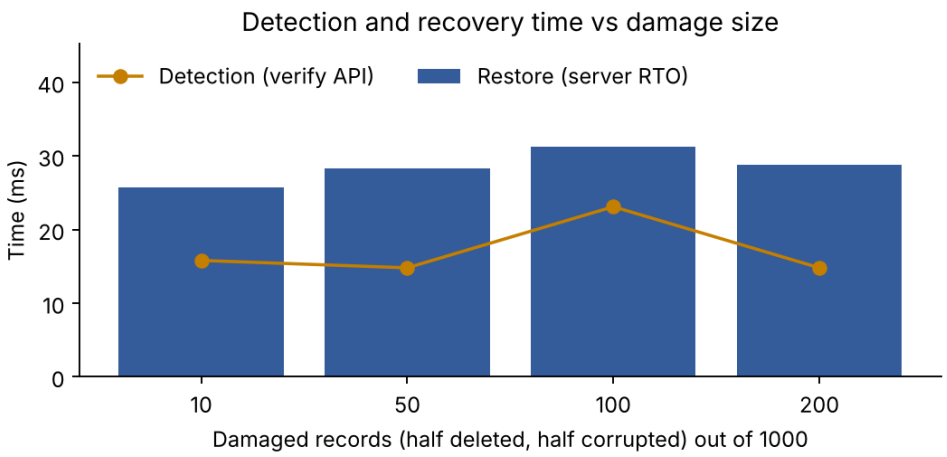


Figure 2 – Detection and recovery time vs number of damaged records

Detection was 100% accurate at every size. Recovery time stays roughly flat (~25–35 ms) because the cost is dominated by downloading and checksumming the backup, not by the number of rows repaired.

8.2 Scenario B – flaky storage (50% transient failures)

Job	Status	Job attempts	Storage retries	Recovery time (ms)
1	COMPLETED	1	1	109.3
2	COMPLETED	1	1	134.3
3	COMPLETED	3	0	3646.6
4	COMPLETED	4	0	7740.6
5	COMPLETED	1	1	100.4

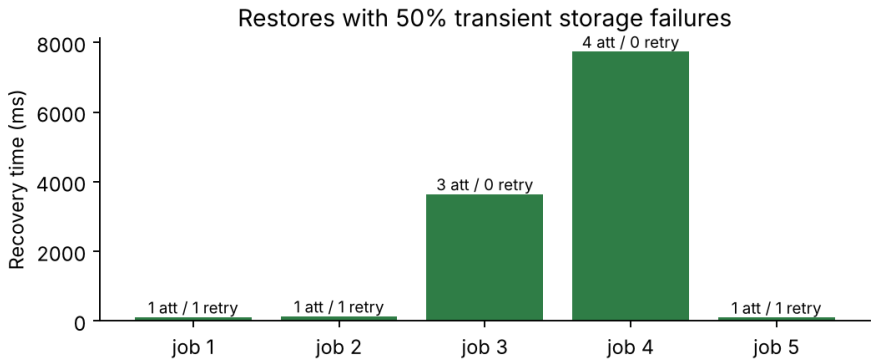


Figure 3 – Restores while half of all storage calls fail

All 5 jobs completed and verified. Three jobs succeeded after one in-call retry (~100–135 ms). In two jobs several consecutive failures opened the circuit breaker; instead of failing, the job was re-queued and succeeded on attempt 3 and 4 (3.6 s and 7.7 s). Dataset identical to baseline: **True**.

8.3 Scenario C – full storage outage for 6 seconds (mandatory demo failure)

Measure	Result
Restore job state history	RETRY_SCHEDULED → COMPLETED
Job attempts until success	4
Recovery time (queued → verified)	7.60 s
Circuit breaker states observed	CLOSED, OPEN
/health states observed	degraded
Read requests during outage	52 succeeded / 0 failed
Dataset identical to baseline	True

The breaker opened after repeated failures, so the worker stopped hammering the dead dependency. The job was kept in the queue (RETRY_SCHEDULED) with backoff, and resumed automatically once storage returned. Users never saw an error: the API degraded gracefully (health = degraded) but kept serving data.

8.4 Scenario D – corrupted backup file → failover

Measure	Result
Requested backup	bkp_20261012092901_a0a58d
Verify result	CORRUPTED
Backup actually used	bkp_20261012092836_d9b89b
Job status	COMPLETED
Recovery time (ms)	25.4
Dataset identical to baseline	True

Bytes were flipped inside the newest backup file. The checksum check rejected it (a restore from it would have written garbage), and the worker failed over to the previous intact backup.

8.5 Proof that data is not corrupted after failures

After every scenario the full dataset fingerprint was recomputed and compared to the baseline: **identical in 100% of runs** (A×4, B, C, D). Additionally each restore job ends with an integrity scan of every record (checksum recomputation) and only reports COMPLETED with verified = 1 if zero records are missing or

corrupted. The restore itself runs in a single transaction, so a crash mid-restore rolls back completely.

8.6 Collected metrics (from GET /metrics at the end of the experiment)

Metric	Value
Recovery time (all restores)	avg 1769.7 ms, p50 100.4 ms, max 7740.6 ms (n=11)
Backup creation time	avg 6.2 ms
Verification time	avg 13.5 ms
Restores successful / failed	11 / 0
Storage retries (in-call)	14
Restore job re-queues	8
Circuit opened / rejected calls	5 / 8
Backup failovers	1
Idempotent replays	1
HTTP successful / failed requests	1251 / 0

9. Load Testing

scripts/load_test.py sends a realistic mix (60% list records, 15% get record, 20% create record, 5% list backups) with an async client at three workload levels against a single API instance (1 worker, local).

Level	Users	Requests	Throughput (req/s)	Avg (ms)	p50	p95	p99	Errors
low	5	500	488.1	10.1	9.1	18.4	25.5	0 (0.0%)
medium	25	2000	380.5	63.6	49.7	161.6	223.3	0 (0.0%)
high	100	5000	257.7	379.0	226.0	1235.4	2543.7	0 (0.0%)

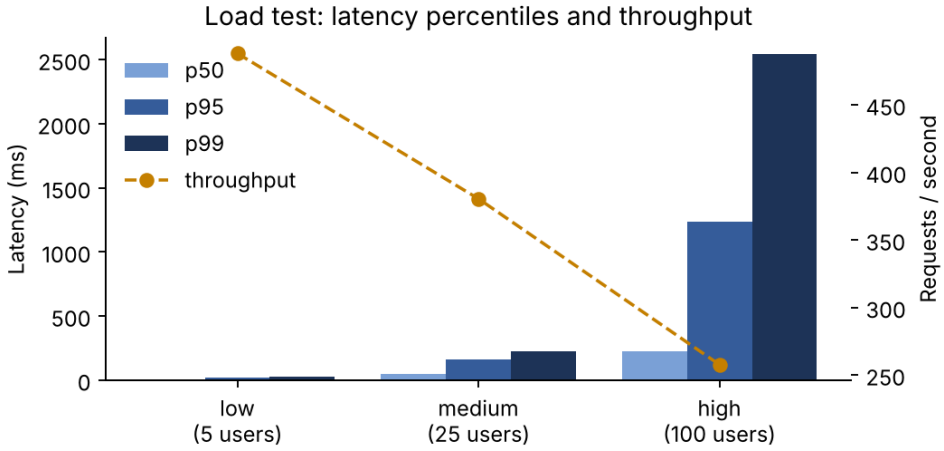


Figure 4 – Latency percentiles and throughput at low / medium / high load

Analysis. No request failed at any level. Latency grows with concurrency (p95 18 → 162 → 1235 ms) and throughput drops from 488 to 258 req/s at 100 users because a single process serialises writes to the SQLite file and the event loop is shared by all users. This is the expected bottleneck; the API is stateless, so in the cloud it scales horizontally (more containers behind the load balancer) once the database is moved to a managed SQL service that handles concurrent writers.

10. Logging

Every important action is written as one JSON line to *logs/app.log* and to the *events* table (queried by GET /logs). Excerpt of warning/error events recorded during the experiment:

```
09:28:45.575 WARNING circuit_state_change   {"breaker": "object-storage", "old": "HALF_OPEN", "new": "OPEN"}
09:28:45.576 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 1, "wait_s": 0.
09:28:45.635 WARNING restore_deferred       {"job_id": "rst_c1dbcc352f", "attempt": 3, "retry_in_s": 4.0, "error": "circui
09:28:49.690 WARNING failure_injected       {"type": "corrupt_records", "corrupted_ids": [89, 265, 346, 464, 526, 554, 641
09:28:49.697 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 1, "wait_s": 0.
09:28:49.908 WARNING failure_injected       {"type": "clear"}
09:28:49.952 WARNING failure_injected       {"type": "corrupt_records", "corrupted_ids": [72, 120, 165, 186, 188, 190, 195
09:28:49.956 WARNING failure_injected       {"type": "storage_outage", "outage_seconds": 6.0}
09:28:49.962 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 1, "wait_s": 0.
09:28:50.030 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 2, "wait_s": 0.
09:28:50.150 WARNING circuit_state_change   {"breaker": "object-storage", "old": "CLOSED", "new": "OPEN"}
09:28:50.151 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 3, "wait_s": 0.
09:28:50.404 WARNING restore_deferred       {"job_id": "rst_ff528089c1", "attempt": 1, "retry_in_s": 1.0, "error": "circui
09:28:51.406 WARNING restore_deferred       {"job_id": "rst_ff528089c1", "attempt": 2, "retry_in_s": 2.0, "error": "circui
09:28:53.409 WARNING circuit_state_change   {"breaker": "object-storage", "old": "HALF_OPEN", "new": "OPEN"}
09:28:53.409 WARNING retry                  {"op": "get:backups/bkp_20261012092836_d9b89b.json", "attempt": 1, "wait_s": 0.
09:28:53.497 WARNING restore_deferred       {"job_id": "rst_ff528089c1", "attempt": 3, "retry_in_s": 4.0, "error": "circui
09:29:01.120 WARNING failure_injected       {"type": "corrupt_backup", "backup_id": "bkp_20261012092901_a0a58d"}
09:29:01.124 ERROR   backup_corruption_detected {"backup_id": "bkp_20261012092901_a0a58d"}
09:29:01.132 WARNING failure_injected       {"type": "delete_records", "deleted_ids": [43, 51, 214, 296, 455, 478, 530, 55
09:29:01.138 ERROR   backup_rejected         {"job_id": "rst_4905ce0bdd", "backup_id": "bkp_20261012092901_a0a58d", "error".
09:29:01.143 WARNING backup_failover        {"job_id": "rst_4905ce0bdd", "requested": "bkp_20261012092901_a0a58d", "used":.
```

11. API Execution Output (captured responses)

The following request/response pairs were captured from the running service during the demo flow (results/api_samples.json). Long fields are shortened.

Login (admin) POST /auth/login → HTTP 200


```
{
 "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIi...",
 "token_type": "bearer",
 "role": "admin"
}
```


Unauthenticated request rejected GET /records → HTTP 401


```
{
 "detail": "missing bearer token"
}
```


Viewer blocked from chaos (RBAC) POST /chaos → HTTP 403


```
{
 "detail": "requires role 'admin'"
}
```


Create backup POST /backups → HTTP 201


```
{
 "id": "bkp_20261012093040_f4e74a",
 "created_at": "2026-10-12T09:30:40.697775+00:00",
 "created_by": "admin",
 "status": "COMPLETED",
 "record_count": 2485,
 "size_bytes": 653156,
 "object_key": "backups/bkp_20261012093040_f4e74a.json",
 "manifest_checksum": "7639d46c0c289547cb503b40c1e2e8cc358f147b5a3d2...",
 "idempotency_key": "demo-capture-1",
 "duration_ms": 13.257796999994298,
 "last_verified_at": null,
 "last_verify_status": null,
 "idempotent_replay": false
}
```


Inject: corrupt 5 records POST /chaos → HTTP 200


```
{
 "type": "corrupt_records",
 "corrupted_ids": [
  918,
  1003,
  1621,
  1953,
  2107
 ]
}
```


Inject: delete 5 records POST /chaos → HTTP 200


```
{
 "type": "delete_records",
 "deleted_ids": [
  334,
  609,
  1107,
  1269,
  1888
 ]
}
```

Verify backup vs live DB POST /backups/bkp_20261012093040_f4e74a/verify → HTTP 200


```
{
 "backup_id": "bkp_20261012093040_f4e74a",
 "backup_integrity": "OK",
 "status": "DAMAGE_DETECTED",
 "records_in_backup": 2485,
 "live_records": 2480,
 "missing": [
  334,
  609,
  1107,
  1269,
  1888
 ],
 "corrupted": [
  918,
  1003,
  1621,
  1953,
  2107
 ],
 "changed_since_backup": 0,
 "new_since_backup": 0,
 "verify_ms": 22.87
}
```


Start repair restore (queued) POST /restores → HTTP 202


```
{
 "id": "rst_17b74f0952",
 "backup_id": "bkp_20261012093040_f4e74a",
 "used_backup_id": null,
 "mode": "repair",
 "fallback": 1,
 "status": "QUEUED",
 "requested_by": "admin",
 "idempotency_key": null,
 "queued_at": "2026-10-12T09:30:40.751644+00:00",
 "started_at": null,
 "finished_at": null,
 "recovery_time_ms": null,
 "attempts": 0,
 "storage_retries": 0
}
```


Restore job status GET /restores/rst_17b74f0952 → HTTP 200


```
{
 "id": "rst_17b74f0952",
 "backup_id": "bkp_20261012093040_f4e74a",
 "used_backup_id": "bkp_20261012093040_f4e74a",
 "mode": "repair",
 "fallback": 1,
 "status": "COMPLETED",
 "requested_by": "admin",
 "idempotency_key": null,
 "queued_at": "2026-10-12T09:30:40.751644+00:00",
 "started_at": "2026-10-12T09:30:40.753122+00:00",
 "finished_at": "2026-10-12T09:30:40.807425+00:00",
 "recovery_time_ms": 55.27645400002257,
 "attempts": 1,
 "storage_retries": 0
}
```


Verify after recovery POST /backups/bkp_20261012093040_f4e74a/verify → HTTP 200


```
{
 "backup_id": "bkp_20261012093040_f4e74a",
 "backup_integrity": "OK",
 "status": "HEALTHY",
 "records_in_backup": 2485,
 "live_records": 2485,
 "missing": [],
 "corrupted": [],
 "changed_since_backup": 0,
 "new_since_backup": 0,
 "verify_ms": 39.86
}
```

Health check GET /health → HTTP 200


```
{
 "status": "ok",
 "database": "ok",
 "object_storage": "ok",
 "circuit_breaker": "CLOSED",
 "queue_depth": 0
}
```


Metrics GET /metrics → HTTP 200


```
{
 "counters": {
  "http_requests_total": 8766,
  "http_success_total": 8766,
  "backups_created_total": 3,
  "idempotent_replays_total": 1,
  "chaos_delete_records_total": 6,
  "chaos_corrupt_records_total": 12,
  "verifications_total": 10,
  "damage_detected_total": 5,
  "restore_jobs_total": 12,
  "restore_success_total": 12
 },
 "circuit_breaker": "CLOSED",
 "queue_depth": 0,
 "timings_ms": {
  "recovery_time": {
   "count": 12,
   "avg": 1626.84,
   "p50": 100.36,
   "p95": 7740.61,
   "max": 7740.61
  }
 }
}
```


12. Deployment

The service is packaged as a Docker image (*Dockerfile*, health check on /health, config via environment variables, data on a mounted volume). Two deployment paths are provided:

- **Render (cloud):** *render.yaml* blueprint – a web service built from the Dockerfile, auto-generated JWT secret, health-check path /health. Deploy: push repo to GitHub → Render → New Blueprint.
- **docker compose:** API container + MinIO (S3-compatible object storage) – the backup files go to a real S3 API bucket, the same code works against AWS S3 by setting BACKUP_BUCKET and AWS credentials.
- Secrets (JWT_SECRET, user passwords) come from environment variables, never from the code.

13. Source Code and Git History

Commit	Date	Message
8c8c5f1	2026-10-12	Project scaffold: dependencies and gitignore
9847ee0	2026-10-12	Data model: users, records, backups, restore_jobs, events (SQLite, transactional)
21e4351	2026-10-12	Structured JSON logging and in-memory metrics
f671f7f	2026-10-12	Resilience: retry with exponential backoff and circuit breaker
9c0c43b	2026-10-12	Object storage layer (local bucket / S3) with fault injection
0d8daac	2026-10-12	JWT authentication with role-based access control
4fa5f9f	2026-10-12	Backup creation, checksum verification and queued atomic restore with failover
a29bd44	2026-10-12	REST API endpoints, chaos injection and health/metrics/logs
488b7bc	2026-10-12	Failure-injection experiment: deletion, corruption, flaky storage, outage, corrupted backup

Commit	Date	Message
3fbf027	2026-10-12	Async load test at low/medium/high workload
a0ed4ed	2026-10-12	Containerise and add Render blueprint + MinIO compose for cloud deployment
e112eb6	2026-10-12	Cloud architecture diagram
6d5fcf1	2026-10-12	Experiment results and charts
9deefe6	2026-10-12	README with run/deploy/demo steps and API documentation

14. Limitations and Future Work

- SQLite allows one writer at a time; move to managed PostgreSQL (RDS / Cloud SQL) for multi-instance scaling.
- The job queue is in-process; a managed queue (SQS) would survive container restarts.
- Backups are full snapshots; incremental backups and scheduled backups (cron) would reduce cost.
- Add backup encryption at rest (S3 SSE / KMS) and cross-region replication for disaster recovery.
- Export metrics to Prometheus/CloudWatch with alerting on damage_detected and circuit_open events.

15. Conclusion

The system meets every project-specific and common requirement: it creates verifiable backups, deliberately deletes and corrupts data and breaks its own storage dependency, detects every fault, and recovers using retry, circuit breaker, queueing, idempotency, failover and graceful degradation. Recovery is measured (tens of milliseconds for data damage, ~7.6 s through a 6 s outage), logged, and proven correct by dataset fingerprints identical to the pre-failure baseline.